

Producción de Software

Pablo Valenzuela

11 Mayo 2026

Gestión de vulnerabilidades en la cadena de suministro de software

1. Por qué ya no basta con analizar solo el código propio
2. Qué es la cadena de suministro de software
3. De encontrar vulnerabilidades a gestionar exposición
4. Areas de seguridad: ordenar el proceso con P-SSCRM
5. Qué están adoptando realmente las organizaciones
6. Transformar evidencia en decisiones
7. Cierre: de herramientas aisladas a gestión continua



¿Dónde puede entrar una vulnerabilidad en una aplicación moderna?

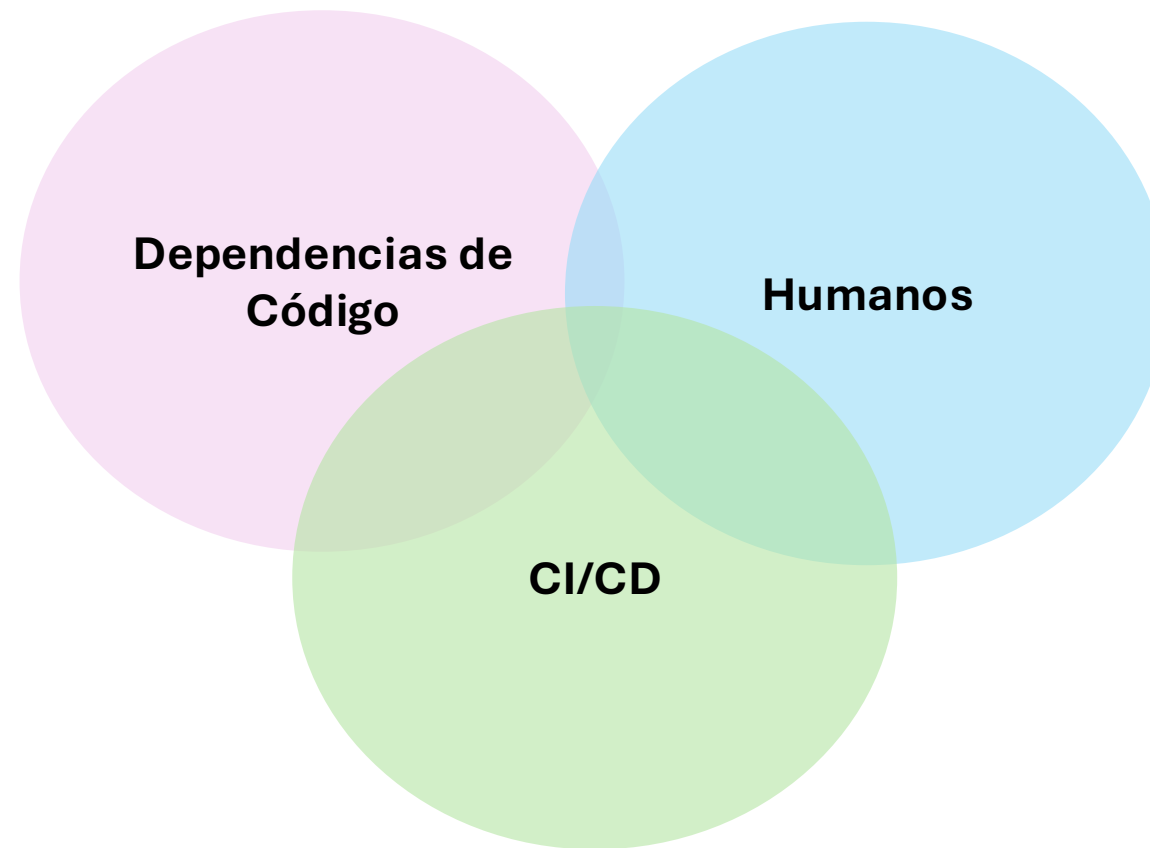


¿Qué diferencia hay entre una vulnerabilidad introducida accidentalmente por un desarrollador y una vulnerabilidad introducida deliberadamente a través de una dependencia o del proceso de build?



Cuando una organización usa componentes externos para acelerar el desarrollo, ¿está reduciendo costos o está desplazando parte del riesgo hacia una zona menos visible?

1. Por qué ya no basta con analizar solo el código propio



2. Qué es la cadena de suministro de software



¿Dónde termina realmente el “software” que desarrolla un equipo: en el código fuente, en las dependencias, en el artefacto compilado, en el contenedor, en el pipeline o en el sistema desplegado?

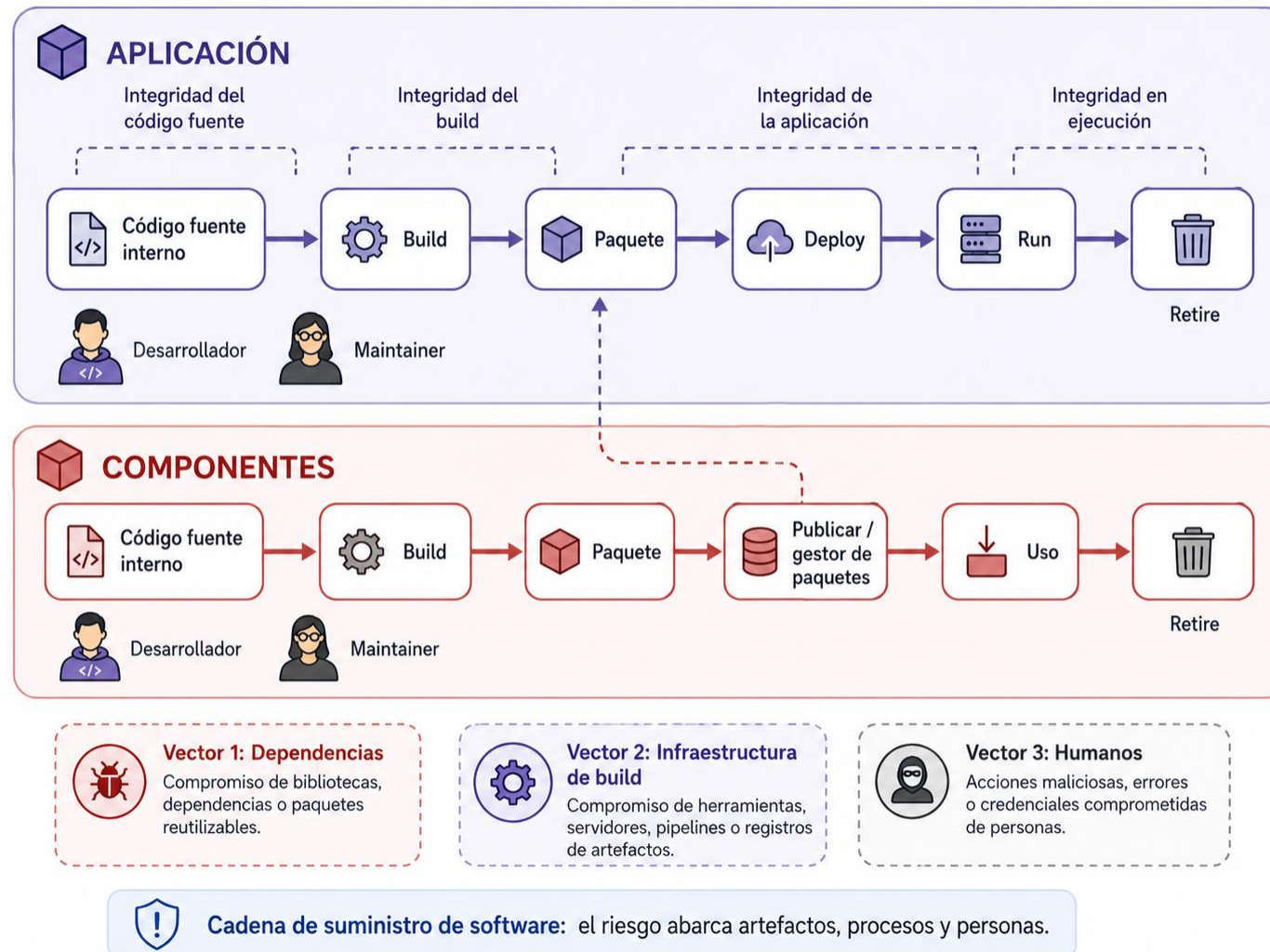


Si dos equipos usan el mismo código fuente, pero distintos pipelines, imágenes base, secretos y dependencias transitivas, ¿producen el mismo software desde el punto de vista de seguridad?



¿Qué elementos de la cadena de suministro son más difíciles de observar: los componentes, los procesos automatizados o las decisiones humanas? ¿Por qué?

2. Qué es la cadena de suministro de software



3. De encontrar vulnerabilidades a gestionar exposición



Si CodeQL reporta una vulnerabilidad crítica en código propio y el SBOM reporta una vulnerabilidad crítica en una dependencia transitiva, ¿cuál debería atenderse primero? ¿Qué información falta para decidir?

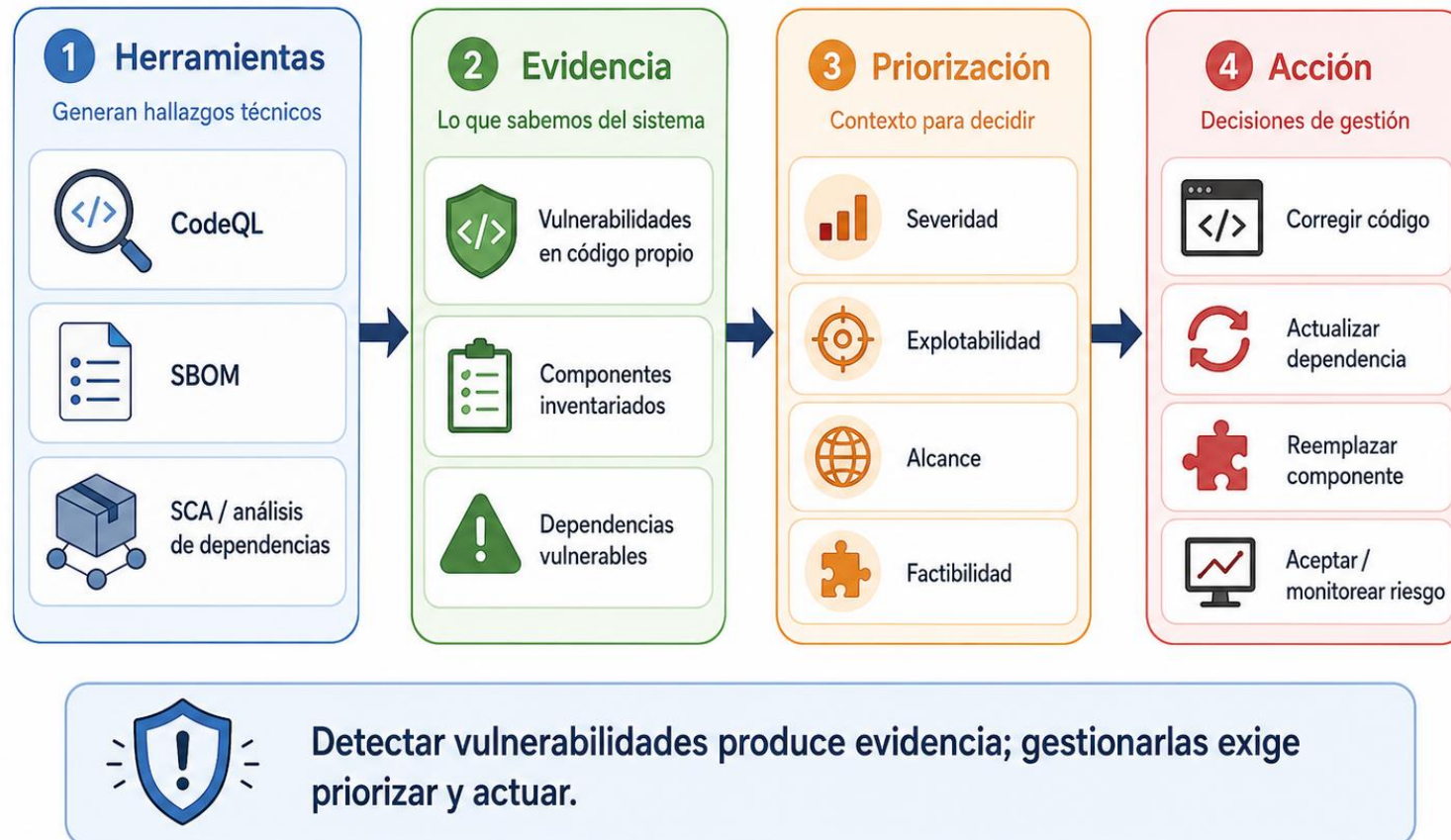


¿Puede una vulnerabilidad reportada por una herramienta no representar un riesgo real para el sistema? ¿Qué condiciones tendrían que cumplirse para justificar no corregirla inmediatamente?



¿Qué cambia cuando dejamos de pensar en vulnerabilidades como “hallazgos” y empezamos a pensarlas como “exposición gestionable”?

3. De encontrar vulnerabilidades a gestionar exposición



4.- Areas de seguridad: ordenar el proceso con P-SSCRM



Si CodeQL reporta una vulnerabilidad crítica en código propio y el SBOM reporta una vulnerabilidad crítica en una dependencia transitiva, ¿cuál debería atenderse primero? ¿Qué información falta para decidir?



¿Puede una vulnerabilidad reportada por una herramienta no representar un riesgo real para el sistema? ¿Qué condiciones tendrían que cumplirse para justificar no corregirla inmediatamente?

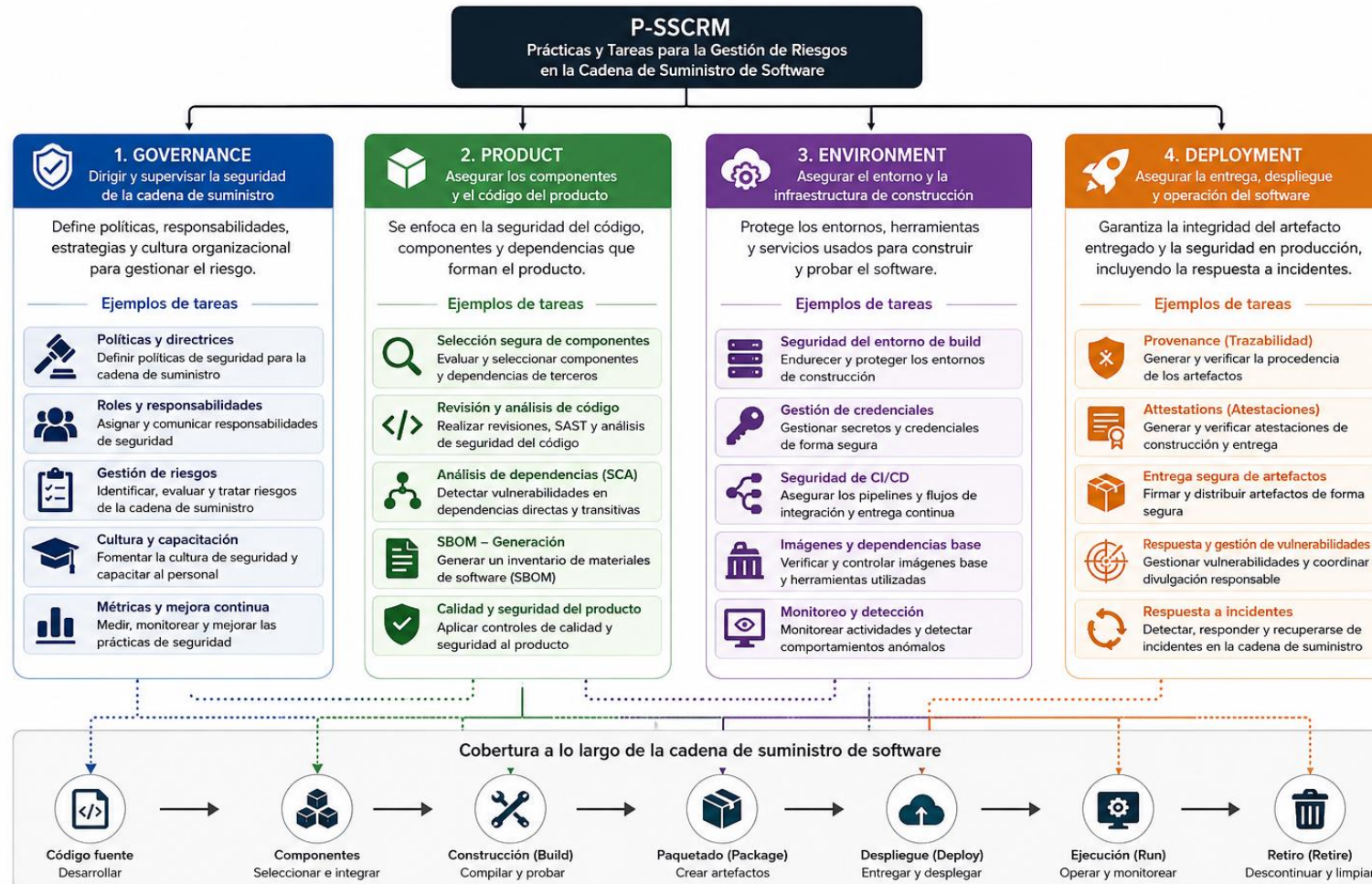


¿Qué cambia cuando dejamos de pensar en vulnerabilidades como “hallazgos” y empezamos a pensarlas como “exposición gestionable”?

4.- Areas de seguridad: ordenar el proceso con P-SSCRM

P-SSCRM: Marco de tareas para la seguridad de la cadena de suministro de software

73 tareas organizadas en 4 grupos principales que cubren todo el ciclo de vida del software



5. Qué están adoptando realmente las organizaciones



¿Por qué algunas prácticas de seguridad se adoptan más rápido que otras?



¿Por qué podría ser difícil implementar tareas como el uso de SBOM, la trazabilidad del origen de los artefactos o los builds reproducibles?

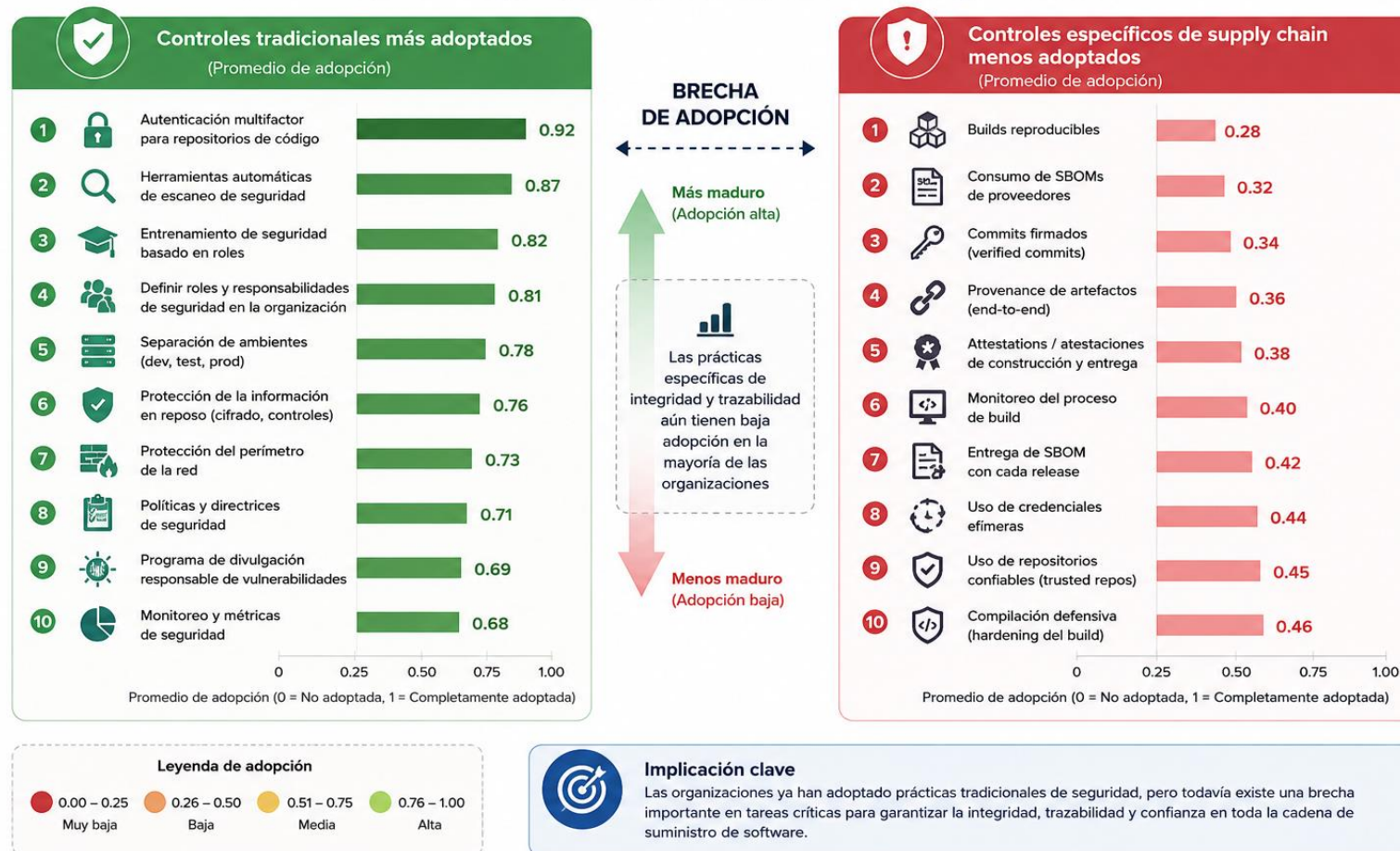


¿Qué riesgos quedan sin cubrir cuando una organización solo aplica controles tradicionales?

5. Qué están adoptando realmente las organizaciones

Brecha de adopción de tareas de seguridad en la cadena de suministro de software

Hallazgos de estudio empírico (P-SSCRM): las organizaciones están más maduras en controles tradicionales, pero tienen baja adopción en prácticas específicas a riesgos modernos.



6.- Actividad aplicada: transformar evidencia en decisiones



Tienen tres hallazgos: una vulnerabilidad en código propio, una dependencia vulnerable y un pipeline con permisos excesivos. Solo pueden intervenir uno esta semana. ¿Qué criterio usarían para decidir y cómo defenderían esa decisión ante el equipo?



Si actualizar una dependencia vulnerable puede romper el sistema, pero mantenerla expone a los usuarios, ¿cómo se justifica una decisión responsable?



¿Qué evidencia permitiría distinguir entre una vulnerabilidad que debe corregirse de inmediato, una que puede monitorearse y una que puede aceptarse temporalmente como riesgo?

6.- Transformar evidencia en decisiones

Canvas de decisión de riesgo en la cadena de suministro de software

Clasifica la evidencia en las tres zonas de la cadena y toma una decisión prioritaria basada en riesgo, impacto y factibilidad.

1. CÓDIGO PROPIO

Lo que desarrollamos

EVIDENCIA (ejemplos)

- CodeQL: SQL Injection en /orders/api.py línea 87 **CRÍTICA**
- CodeQL: Deserialización insegura en ReportService.java **ALTA**
- Falta validación de permisos en endpoint /admin/users **ALTA**
- Cobertura de pruebas: 45% en módulo crítico **MEDIA**
- Code review pendiente en módulo de pagos **BAJA**

CONSIDERACIONES CLAVE

- Tenemos control directo para corregir.
- El fix puede requerir cambios de código y pruebas.
- Conocemos el contexto y el flujo de la aplicación.

DECISIONES POSIBLES

- ☐ Priorizar corrección inmediata
- ☐ Planificar corrección en siguiente iteración
- ☐ Aceptar temporalmente y mitigar
- ☐ Monitorear (no priorizar por ahora)

2. DEPENDENCIAS

Componentes que consumimos

EVIDENCIA (ejemplos)

- SCA: log4j 1.2.17 – CVE-2021-44228 – Crítica **CRÍTICA**
- SCA: jackson-databind 2.9.8 – CVE-2020-36518 – Alta **ALTA**
- SCA: openssl 1.1.1k – CVE-2021-3711 – Media **MEDIA**
- Dependencia transitiva a través de librería X **MEDIA**
- Licencia incompatible en componente Y **BAJA**

CONSIDERACIONES CLAVE

- No tenemos control directo sobre el código.
- Actualizar puede romper compatibilidades.
- Puede haber mitigaciones o parches disponibles.

DECISIONES POSIBLES

- ☐ Actualizar/Remediar pronto
- ☐ Evaluar actualización con pruebas
- ☐ Aceptar temporalmente y mitigar
- ☐ Monitorear (con alerta y revisión periódica)

3. PIPELINE / BUILD

Cómo construimos y entregamos

EVIDENCIA (ejemplos)

- Token de GitHub con permisos de escritura global **CRÍTICA**
- Secrets almacenados en variables de entorno **ALTA**
- Workflow sin pin de versiones de acciones **ALTA**
- Runner auto-hospedado sin aislamiento **MEDIA**
- Sin verificación de integridad de artefactos **MEDIA**

CONSIDERACIONES CLAVE

- Compromete todo el software, no solo un módulo.
- Puede ser explotado para inyectar código malicioso.
- La corrección puede requerir cambios de proceso.

DECISIONES POSIBLES

- ☐ Corregir configuración de inmediato
- ☐ Mejorar controles en el corto plazo
- ☐ Aceptar temporalmente y mitigar
- ☐ Monitorear (con detección y alertas)

TOMA DE DECISIÓN PRIORITARIA

Solo podemos intervenir en UN elemento esta semana.

	1. Código propio	2. Dependencias	3. Pipeline / Build
Impacto potencial	4	5	5
Explotabilidad	3	4	5
Alcance	3	4	5
Factibilidad de remediación	3	2	2
Confianza en detección	3	4	3
PUNTUACIÓN TOTAL (1-5 cada criterio)	16 / 25	19 / 25	20 / 25

➔

DECISIÓN RECOMENDADA

Priorizar:

3. PIPELINE / BUILD

Justificación:

El compromiso del pipeline tendría impacto y alcance máximos, y es altamente explotable. Aunque la remediación requiere cambios de proceso, reduce el riesgo sistémico en toda la cadena.

CRITERIOS DE EVALUACIÓN (1-5)

1 = Muy bajo 2 = Bajo 3 = Medio 4 = Alto 5 = Muy alto

- Impacto potencial:** ¿Qué tan grave sería si se explota?
- Explotabilidad:** ¿Qué tan fácil es que sea explotado?
- Alcance:** ¿A cuántos usuarios, sistemas o datos afecta?
- Factibilidad de remediación:** ¿Qué tan costoso o complejo es corregirlo?
- Confianza en detección:** ¿Qué tan seguros estamos de que podamos detectar actividad maliciosa relacionada?

RECUERDA

La prioridad puede cambiar con nueva información. Revisa y reevalúa periódicamente.

🔍 Detectar → 📋 Evaluar → 🎯 Priorizar → 🔑 Remediar → 🛡️ Verificar → 👁️ Monitorear

7.- De herramientas aisladas a gestión continua



¿Qué tendría que pasar para que la gestión de vulnerabilidades deje de ser una actividad reactiva y se convierta en una capacidad continua del equipo?



¿Cómo se puede demostrar que una vulnerabilidad fue realmente gestionada y no solo “cerrada” en una herramienta?



¿Qué evidencia debería conservar una organización para explicar, semanas o meses después, por qué priorizó ciertas vulnerabilidades y no otras?

Ciclo continuo de gestión de vulnerabilidades en la cadena de suministro de software

Un proceso iterativo y basado en evidencia para reducir la exposición y aumentar la confianza en cada entrega.



EVIDENCIA DE CONFIANZA

La confianza se construye y se demuestra con evidencia verificable a lo largo de toda la cadena.



SBOMs actualizados



Escaneos y reportes de vulnerabilidades



Firmas, attestaciones y provenance



Registros del pipeline y builds



Decisiones de riesgo y remediaciones



Métricas y tendencias en el tiempo



RESULTADO ESPERADO

Menor exposición, mayor resiliencia y capacidad de responder rápidamente ante nuevos riesgos.

Referencias



Williams, L., & Migue, S. (2025). *Establishing a baseline of software supply chain security task adoption by software organizations*. In **Proceedings of the 2025 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '25)**. Association for Computing Machinery.